

Absolute Value Program (Copy-paste code)

```
#include <iostream>
using namespace std;

template <typename T>

/** Returns the absolute value of the argument.
 * The function checks if the argument is negative,
 * and if it is, it multiplies it by -1 to make the
 * returned value positive. If the argument is already
 * positive, return the value as normal. */
T absolute(T a) {
    if (a < 0) {
        return a * -1;
    }
    return a;
}

int main() {
    double d = -1.2;
    int i = -6;

    cout << absolute(d) << endl;
    cout << absolute(i) << endl;
    return 0;
}
```

Screenshot of Editor:

```
1  ▶ #include <iostream>
2    using namespace std;
3
4    template <typename T>
5
6    /** Returns the absolute value of the argument.
7     * The function checks if the argument is negative,
8     * and if it is, it multiplies it by -1 to make the
9     * returned value positive. If the argument is already
10    * positive, return the value as normal. */
11    T absolute(T a) {
12        if (a < 0) {
13            return a * -1;
14        }
15        return a;
16    }
17
18  ▶ int main() {
19      double d = -1.2;
20      int i = -6;
21
22      cout << absolute(d) << endl;
23      cout << absolute(i) << endl;
24      return 0;
25  }
```

Array Index Program (Copy-paste code)

```
#include <iostream>
using namespace std;

int getElement(const int[], int, int);

int main() {
    const int size = 5;
    int arr[size] = {10, 20, 30, 40, 50};

    while (true) {
        cout << "Enter an index to access (negative value to quit): ";
        int index;
        cin >> index;

        if (index < 0) {
            break;
        }

        try {
            int value = getElement(arr, size, index);
            cout << "Element at index " << index << " is " << value << endl;
        } catch (const std::out_of_range& e) {
            cerr << "Error: " << e.what() << std::endl;
        }
    }
    return 0;
}

int getElement(const int a[], int size, int index) {
    // Throw error if requested position is out of bounds
    if (index > (size - 1)) { // Array indices end at size - 1
        throw std::out_of_range("Array index out of bounds");
    }
    return a[index];
}
```

Screenshot of Editor:

```
1  ▶ #include <iostream>
2    using namespace std;
3
4  ↵ int getElement(const int[], int, int);
5
6  ▶ int main() {
7      const int size = 5;
8      int arr[size] = {10, 20, 30, 40, 50};
9
10     while (true) {
11         cout << "Enter an index to access (negative value to quit): ";
12         int index;
13         cin >> index;
14
15         if (index < 0) {
16             break;
17         }
18
19         try {
20             int value = getElement(arr, size, index);
21             cout << "Element at index " << index << " is " << value << endl;
22         } catch (const std::out_of_range& e) {
23             cerr << "Error: " << e.what() << std::endl;
24         }
25     }
26     return 0;
27 }
```

```
28
29 ↵ int getElement(const int a[], int size, int index) {
30     // Throw error if requested position is out of bounds
31     if (index > (size - 1)) { // Array indices end at size - 1
32         throw std::out_of_range("Array index out of bounds");
33     }
34     return a[index];
35 }
```

Output:

```
1.2
```

```
6
```

```
Process finished with exit code 0
```

```
Enter an index to access (negative value to quit):0
```

```
Element at index 0 is 10
```

```
Enter an index to access (negative value to quit):1
```

```
Element at index 1 is 20
```

```
Enter an index to access (negative value to quit):2
```

```
Element at index 2 is 30
```

```
Enter an index to access (negative value to quit):3
```

```
Element at index 3 is 40
```

```
Enter an index to access (negative value to quit):4
```

```
Element at index 4 is 50
```

```
Enter an index to access (negative value to quit):5
```

```
Error: Array index out of bounds
```

```
Enter an index to access (negative value to quit):-1
```

```
Process finished with exit code 0
```